

Aarya Birwadkar

Batch C2 PRN-1262252502

Data Structures Assignment 6

Codes:

1) INFIX TO POSTFIX:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX_SIZE 50
```

```
char stack[MAX_SIZE];
```

```
int top = -1;
```

```
int isEmpty()
```

```
{
```

```
    if (top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
int isFull()
```

```
{
```

```
    if (top == MAX_SIZE - 1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
void push(char item)
{
    if (!isFull())
    {
        top = top + 1;
        stack[top] = item;
    }
}
```

```
char pop()
{
    char temp;

    if (!isEmpty())
    {
        temp = stack[top];
        top = top - 1;
        return temp;
    }

    return '\0';
}
```

```
int isp(char ch)
{
    switch (ch)
    {
        case '+':
```

```
    case '-':  
        return 1;  
  
    case '*':  
    case '/':  
    case '%':  
        return 2;  
  
    case '^':  
        return 3;  
  
    case '(':  
        return 0;  
  
    default:  
        return 0;  
    }  
}
```

```
int icp(char ch)  
{  
    switch (ch)  
    {  
        case '+':  
        case '-':  
            return 1;  
  
        case '*':  
        case '/':
```

```

        case '%':
            return 2;

        case '^':
            return 4;

        case '(':
            return 5;

        default:
            return 0;
    }
}

```

```

void in_post(char inexp[], char postexp[])
{
    int k = 0, i = 0;
    char tkn;

    tkn = inexp[i];

    while (tkn != '\0')
    {
        if (isalnum(tkn))
        {
            postexp[k] = tkn;
            k++;
        }
        else

```

```

{
    if (tkn == '(')
    {
        push(tkn);
    }
    else if (tkn == ')')
    {
        while (!isEmpty() && stack[top] != '(')
        {
            postexp[k] = pop();
            k++;
        }

        if (!isEmpty() && stack[top] == '(')
            pop();
    }
    else
    {
        while (!isEmpty() && isp(stack[top]) >= icp(tkn))
        {
            postexp[k] = pop();
            k++;
        }

        push(tkn);
    }
}

i++;

```

```

        tkn = inexp[i];
    }

    while (!isEmpty())
    {
        postexp[k] = pop();
        k++;
    }

    postexp[k] = '\0';
}

int main()
{
    char inexp[50], postexp[50];

    printf("Infix Expression: ");
    scanf("%49s", inexp);

    in_post(inexp, postexp);

    printf("Postfix Expression = %s\n", postexp);

    return 0;
}

```

Output:

dell@dell-Latitude-5420:~\$ gcc ass6.c

dell@dell-Latitude-5420:~\$./a.out

Infix Expression: a*(b+c)-d/e

Postfix Expression = abc+*de/-

2) INFIX TO PREFIX:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 50
```

```
char stack[MAX_SIZE];
```

```
int top = -1;
```

```
int isEmpty()
```

```
{
```

```
    if (top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
int isFull()
```

```
{
```

```
    if (top == MAX_SIZE - 1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
void push(char item)
```

```
{
```

```
    if (!isFull())
    {
        top = top + 1;
        stack[top] = item;
    }
}
```

```
char pop()
```

```
{
    char temp;
```

```
    if (!isEmpty())
```

```
    {
        temp = stack[top];
        top = top - 1;
        return temp;
    }
```

```
    return '\0';
```

```
}
```

```
int isp(char ch)
```

```
{
    switch (ch)
    {
        case '+':
        case '-':
            return 1;
```

```
    case '*':  
    case '/':  
    case '%':  
        return 2;  
  
    case '^':  
        return 4;  
  
    case '(':  
        return 0;  
  
    default:  
        return 0;  
    }  
}
```

```
int icp(char ch)  
{  
    switch (ch)  
    {  
        case '+':  
        case '-':  
            return 1;  
  
        case '*':  
        case '/':  
        case '%':  
            return 2;
```

```
    case '^':  
        return 3;  
  
    case '(':  
        return 5;  
  
    default:  
        return 0;  
    }  
}
```

```
void reverse(char str[])  
{  
    int i = 0;  
    int j = strlen(str) - 1;  
    char temp;  
  
    while (i < j)  
    {  
        temp = str[i];  
        str[i] = str[j];  
        str[j] = temp;  
  
        i++;  
        j--;  
    }  
}
```

```
void swapBrackets(char str[])
```

```
{
    int i = 0;

    while (str[i] != '\0')
    {
        if (str[i] == '(')
            str[i] = ')';
        else if (str[i] == ')')
            str[i] = '(';

        i++;
    }
}
```

```
void in_pre(char inexp[], char preexp[])
```

```
{
    int k = 0, i = 0;
    char tkn;

    top = -1;

    reverse(inexp);
    swapBrackets(inexp);

    tkn = inexp[i];

    while (tkn != '\0')
    {
        if (isalnum(tkn))
```

```

{
    preexp[k] = tkn;
    k++;
}
else
{
    if (tkn == '(')
    {
        push(tkn);
    }
    else if (tkn == ')')
    {
        while (!isEmpty() && stack[top] != '(')
        {
            preexp[k] = pop();
            k++;
        }

        if (!isEmpty() && stack[top] == '(')
            pop();
    }
    else
    {
        while (!isEmpty() && isp(stack[top]) > icp(tkn))
        {
            preexp[k] = pop();
            k++;
        }
    }
}

```

```
        push(tkn);
    }
}
```

```
    i++;
    tkn = inexp[i];
}
```

```
while (!isEmpty())
{
    preexp[k] = pop();
    k++;
}
```

```
preexp[k] = '\0';
```

```
reverse(preexp);
}
```

```
int main()
{
    char inexp[MAX_SIZE];
    char preexp[MAX_SIZE];

    printf("Infix Expression: ");
    scanf("%49s", inexp);

    in_pre(inexp, preexp);
}
```

```
printf("Prefix Expression = %s\n", preexp);

return 0;
}
```

Output:

dell@dell-Latitude-5420:~\$ gcc ass6.c

dell@dell-Latitude-5420:~\$./a.out

Infix Expression: (a+b)*c

Prefix Expression = *+abc

3) POSTFIX TO INFIX:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 50
```

```
char stack[MAX_SIZE][MAX_SIZE];
```

```
int top = -1;
```

```
int isEmpty()
```

```
{
```

```
    if (top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
int isFull()
{
    if (top == MAX_SIZE - 1)
        return 1;
    else
        return 0;
}
```

```
void push(char item[])
{
    if (!isFull())
    {
        top = top + 1;
        strcpy(stack[top], item);
    }
}
```

```
void pop(char item[])
{
    if (!isEmpty())
    {
        strcpy(item, stack[top]);
        top = top - 1;
    }
}
```

```
void post_in(char postexp[], char inexp[])
{
    int i = 0;
```

```
char tkn;  
char op1[MAX_SIZE], op2[MAX_SIZE];  
char temp[MAX_SIZE];
```

```
top = -1;
```

```
tkn = postexp[i];
```

```
while (tkn != '\0')
```

```
{
```

```
    if (isalnum(tkn))
```

```
    {
```

```
        temp[0] = tkn;
```

```
        temp[1] = '\0';
```

```
        push(temp);
```

```
    }
```

```
    else
```

```
    {
```

```
        pop(op2);
```

```
        pop(op1);
```

```
        strcpy(temp, "(");
```

```
        strcat(temp, op1);
```

```
        int len = strlen(temp);
```

```
        temp[len] = tkn;
```

```
        temp[len + 1] = '\0';
```

```
        strcat(temp, op2);
```

```

        strcat(temp, "");

        push(temp);
    }

    i++;
    tkn = postexp[i];
}

pop(inexp);

if (inexp[0] == '(')
{
    int len = strlen(inexp);

    if (inexp[len - 1] == ')')
    {
        inexp[len - 1] = '\0';
        memmove(inexp, inexp + 1, strlen(inexp));
    }
}
}

int main()
{
    char postexp[MAX_SIZE];
    char inexp[MAX_SIZE];

    printf("Postfix Expression: ");

```

```

scanf("%49s", postexp);

post_in(postexp, inexp);

printf("Infix Expression = %s\n", inexp);

return 0;
}

```

Output:

```

dell@dell-Latitude-5420:~$ gcc ass6.c
dell@dell-Latitude-5420:~$ ./a.out
Postfix Expression: abc*+
Infix Expression = a+(b*c)

```

4) POSTFIX TO PREFIX:

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX_SIZE 50

char stack[MAX_SIZE][MAX_SIZE];
int top = -1;

int isEmpty()
{
    if (top == -1)
        return 1;
}

```

```
    else  
        return 0;  
}
```

```
int isFull()  
{  
    if (top == MAX_SIZE - 1)  
        return 1;  
    else  
        return 0;  
}
```

```
void push(char item[])  
{  
    if (!isFull())  
    {  
        top = top + 1;  
        strcpy(stack[top], item);  
    }  
}
```

```
void pop(char item[])  
{  
    if (!isEmpty())  
    {  
        strcpy(item, stack[top]);  
        top = top - 1;  
    }  
}
```

```

void post_pre(char postexp[], char preexp[])
{
    int i = 0;
    char tkn;
    char op1[MAX_SIZE], op2[MAX_SIZE];
    char temp[MAX_SIZE];

    top = -1;

    tkn = postexp[i];

    while (tkn != '\0')
    {
        if (isalnum(tkn))
        {
            temp[0] = tkn;
            temp[1] = '\0';

            push(temp);
        }
        else
        {
            pop(op2);
            pop(op1);

            temp[0] = tkn;
            temp[1] = '\0';

```

```

        strcat(temp, op1);
        strcat(temp, op2);

        push(temp);
    }

    i++;
    tkn = postexp[i];
}

pop(preexp);
}

int main()
{
    char postexp[MAX_SIZE];
    char preexp[MAX_SIZE];

    printf("Postfix Expression: ");
    scanf("%49s", postexp);

    post_pre(postexp, preexp);

    printf("Prefix Expression = %s\n", preexp);

    return 0;
}

```

Output:

dell@dell-Latitude-5420:~\$ gcc ass6.c

dell@dell-Latitude-5420:~\$./a.out

Postfix Expression: abc*+

Prefix Expression = +a*bc

5) PREFIX TO INFIX:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 50
```

```
char stack[MAX_SIZE][MAX_SIZE];
```

```
int top = -1;
```

```
int isEmpty()
```

```
{
```

```
    if (top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```
int isFull()
```

```
{
```

```
    if (top == MAX_SIZE - 1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

```

void push(char item[])
{
    if (!isFull())
    {
        top = top + 1;
        strcpy(stack[top], item);
    }
}

```

```

void pop(char item[])
{
    if (!isEmpty())
    {
        strcpy(item, stack[top]);
        top = top - 1;
    }
}

```

```

void pre_in(char preexp[], char inexp[])
{
    int i = strlen(preexp) - 1;
    char tkn;
    char op1[MAX_SIZE], op2[MAX_SIZE];
    char temp[MAX_SIZE];

    top = -1;

    while (i >= 0)
    {

```

```

    tkn = preexp[i];

    if (isalnum(tkn))
    {
        temp[0] = tkn;
        temp[1] = '\0';

        push(temp);
    }
    else
    {
        pop(op1);
        pop(op2);

        strcpy(temp, "(");
        strcat(temp, op1);

        int len = strlen(temp);
        temp[len] = tkn;
        temp[len + 1] = '\0';

        strcat(temp, op2);
        strcat(temp, ")");

        push(temp);
    }

    i--;
}

```

```

        pop(inexp);
    }

int main()
{
    char preexp[MAX_SIZE];
    char inexp[MAX_SIZE];

    printf("Prefix Expression: ");
    scanf("%49s", preexp);

    pre_in(preexp, inexp);

    printf("Infix Expression = %s\n", inexp);

    return 0;
}

```

Output:

dell@dell-Latitude-5420:~\$ gcc ass6.c

dell@dell-Latitude-5420:~\$./a.out

Prefix Expression: +a*bc

Infix Expression = (a+(b*c))

6) PREFIX TO POSTFIX:

```

#include <ctype.h>
#include <string.h>

```

```

#define MAX_SIZE 50

```

```

char stack[MAX_SIZE][MAX_SIZE];
int top = -1;

int isEmpty()
{
    if (top == -1)
        return 1;
    else
        return 0;
}

int isFull()
{
    if (top == MAX_SIZE - 1)
        return 1;
    else
        return 0;
}

void push(char item[])
{
    if (!isFull())
    {
        top = top + 1;
        strcpy(stack[top], item);
    }
}

void pop(char item[])
{
    if (!isEmpty())
    {
        strcpy(item, stack[top]);
        top = top - 1;
    }
}

void pre_post(char preexp[], char postexp[])
{
    int i = strlen(preexp) - 1;
    char tkn;
    char op1[MAX_SIZE], op2[MAX_SIZE];
    char temp[MAX_SIZE];

```

```

top = -1;

while (i >= 0)
{
    tkn = preexp[i];

    if (isalnum(tkn))
    {
        temp[0] = tkn;
        temp[1] = '\0';

        push(temp);
    }
    else
    {
        pop(op1);
        pop(op2);

        strcpy(temp, op1);
        strcat(temp, op2);

        int len = strlen(temp);
        temp[len] = tkn;
        temp[len + 1] = '\0';

        push(temp);
    }

    i--;
}

pop(postexp);
}

int main()
{
    char preexp[MAX_SIZE];
    char postexp[MAX_SIZE];

    printf("Prefix Expression: ");
    scanf("%49s", preexp);

```

```
pre_post(preexp, postexp);

printf("Postfix Expression = %s\n", postexp);

return 0;
}
```

Output:

```
dell@dell-Latitude-5420:~$ gcc ass6.c
dell@dell-Latitude-5420:~$ ./a.out
Prefix Expression: +a*bc
Postfix Expression = abc*+
#include <stdio.h>
```